Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИсп1
Лебедев Г. М.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 6
4. Результаты экспериментов и тестирования 9
Заключение 10
Литература 11
Приложение 12

Введение

В рамках лабораторной работы №1 рассматривается реализация двух пользовательских классов: TBitField (битовое поле) и TSet (множество на основе битового поля).

Битовое поле — это компактная структура данных для хранения последовательности битов. В отличие от массива bool (по 1 байту на элемент), битовое поле упаковывает 32 бита в одно 32-битное число unsigned int, что даёт экономию памяти почти в 8 раз.

Множество — математический объект, хранящий уникальные элементы и поддерживающий операции объединения, пересечения, разности и дополнения. Если элементами множества являются целые числа из диапазона 0..N-1, его удобно представить характеристическим вектором — битовым полем длины N, где бит i равен 1, если число i входит в множество.

Цель работы: освоить принципы ООП (инкапсуляцию, перегрузку операторов, правило трёх) и научиться писать автотесты с использованием Google Test.

1. Постановка задачи

Требуется реализовать на языке C++ два класса:

1. Класс TBitField — битовое поле с следующими операциями:
   - конструктор по длине, конструктор копирования, деструктор;
   - методы GetLength, SetBit, ClrBit, GetBit;
   - перегруженные операторы ==, !=, =, |, &, ~;
   - операторы ввода/вывода в поток.

2. Класс TSet — множество на основе TBitField:
   - конструктор по мощности, конструктор копирования, конструктор преобразования из TBitField;
   - оператор преобразования к TBitField;
   - методы GetMaxPower, InsElem, DelElem, IsMember;
   - теоретико-множественные операции: + (с элементом и с множеством), - (с элементом), * (пересечение), ~ (дополнение);
   - операторы сравнения и ввода/вывода.

Объявления классов должны точно соответствовать официальным заголовкам из папки includes/Lab1 репозитория. Реализация выносится в отдельные .cpp-файлы.

Дополнительно требуется написать набор тестов Google Test, покрывающих основной функционал и краевые случаи обоих классов.

2. Руководство пользователя

Данная работа не является консольным приложением с интерактивным вводом. Всё взаимодействие происходит через автотесты Google Test.

Форматы ввода/вывода:

TBitField:
Ввод: строка из нулей и единиц (например, 10110), длина строки равна длине битового поля.
Вывод: строка из нулей и единиц (например, 10110)

TSet:
Ввод: целые числа (номера элементов), разделенные пробелами или запятыми (например,1 3, 5). Элементы, выходящие за пределы универсума, игнорируются.
Вывод: множество в фигурных скобках, элементы разделены запятой и пробелом (например, { 1, 3, 5 }).

При попытке выполнить бинарную операцию (+, *) над множествами с разной мощностью (размером универсума) выбрасывается исключение std::invalid_argument.

Запуск тестов:

1. Собрать проект:
   ```
   $ cd LebedevG/Lab1
   $ mkdir build && cd build
   $ cmake ..
   $ cmake --build .
   ```

2. Запустить исполняемый файл тестов:
   ```
   $ ./lab1_test
   ```

3. На экране отобразится список выполненных тестов и итог:
   ```
   [==========] 21 tests from 2 test suites ran.
   [  PASSED  ] 21 tests.
   ```

Если какой-то тест не прошёл — будет выведено сообщение об ошибке с указанием файла, номера строки, ожидаемого и фактического значений.

Также тесты можно запускать через панель Testing в VS Code (расширение TestMate C++) или через ctest.

3. Руководство программиста

3.1. Описание структуры программы

Проект состоит из следующих модулей:

- tbitfield.h — объявление класса TBitField (официальный заголовок из includes/Lab1, не дублируется в проекте).
- tbitfield.cpp — реализация методов класса TBitField.
- tset.h — объявление класса TSet (официальный заголовок).
- tset.cpp — реализация методов класса TSet.
- test_lab1.cpp — набор тестов Google Test для обоих классов.
- main.cpp — точка входа, инициализирует и запускает Google Test.
- CMakeLists.txt — скрипт сборки. Через FetchContent скачивает Google Test, собирает исполняемый файл lab1_test и подключает официальные заголовки из includes/Lab1 через target_include_directories.

3.2. Описание структур данных

Класс TBitField:

Поля:
- int BitLen — логическая длина (число доступных битов);
- int MemLen — размер массива pMem (сколько TELEM нужно);
- TELEM* pMem — динамический массив элементов типа unsigned int.

Приватные методы:
- GetMemIndex(n) — индекс элемента для бита n: n / 32;
- GetMemMask(n) — маска бита: 1u << (n % 32).

Бит с номером n хранится в элементе pMem[n / 32] на позиции n % 32.
Это даёт плотную упаковку: 32 бита в одном unsigned int.

Класс TSet:

Поля:
- int MaxPower; — мощность универсума (числа 0..MaxPower-1);
- TBitField BitField; — характеристический вектор множества.

Бит i вектора равен 1, если число i входит в множество, и 0 иначе.
Все теоретико-множественные операции сводятся к побитовым операциям над характеристическими векторами.

3.3. Описание алгоритмов

Установка бита (SetBit):

1. Проверить, что 0 <= n < BitLen.
2. Вычислить индекс элемента: idx = n / 32.
3. Вычислить маску: mask = 1u << (n % 32).
4. Выполнить побитовое ИЛИ: pMem[idx] |= mask.

Очистка бита (ClrBit):

1. Проверить границы.
2. idx = n / 32, mask = 1u << (n % 32).
3. pMem[idx] &= ~mask; (обнуляем нужный бит, остальные не трогаем)

Чтение бита (GetBit):

1. Проверить границы (если вне — вернуть 0).
2. Вернуть (pMem[idx] & mask) ? 1 : 0.

Операция ИЛИ (operator|):

1. Взять максимум длин двух полей.
2. Создать новое поле этой длины.
3. Для каждого элемента i: result[i] = a[i] | b[i]. Если в одном из полей элементов меньше — считать их нулями.

Операция И (operator&):

1. Взять максимум длин двух полей.
2. Создать новое поле этой длины.
3. Для каждого элемента i: result[i] = a[i] & b[i].

Инверсия (operator~):

1. Создать результат той же длины.
2. Для каждого элемента: result[i] = ~pMem[i].
3. В последнем элементе затереть «лишние» старшие биты, выходящие за пределы BitLen (через маску ~0u >> extra).

Конструктор копирования:

1. Скопировать поля BitLen и MemLen из исходного объекта.
2. Выделить новый массив pMem той же длины.
3. Поэлементно скопировать содержимое массива.

Оператор присваивания (operator=):

1. Проверить самоприсваивание (this == &bf) — если да, вернуть *this.
2. Освободить старую память (delete[] pMem).
3. Скопировать поля BitLen и MemLen.
4. Выделить новый массив и скопировать данные.
5. Вернуть *this.

Объединение множеств (operator+):

1. Скопировать левое множество.
2. Применить побитовое ИЛИ к характеристическим векторам.
3. Вернуть результат.

Пересечение (operator*):

1. Скопировать левое множество.
2. Применить побитовое И к характеристическим векторам.

Дополнение (operator~):

1. Скопировать множество.
2. Применить инверсию к характеристическому вектору.

Добавление элемента (InsElem):

1. Вызвать SetBit для соответствующего номера в характеристическом векторе.

Удаление элемента (DelElem):

1. Вызвать ClrBit для соответствующего номера в характеристическом векторе.

Проверка принадлежности (IsMember):

1. Вернуть GetBit для соответствующего номера.

Вывод множества (operator<<):

1. Вывести открывающую скобку и пробел.
2. Пройти по всем числам от 0 до MaxPower-1.
3. Если число принадлежит множеству — вывести его, разделяя элементы запятой с пробелом.
4. Вывести закрывающую скобку.

4. Результаты экспериментов и тестирования

Написано 24 тест Google Test, разбитых на два набора.

Набор TBitFieldTest (11 тестов):
- CreateAndLength — проверка корректности длины после создания;
- SetAndGetBit — установка и чтение битов;
- ClearBit — сброс бита;
- OutOfRangeDoesNotCrash — выход за границы не должен приводить к падению программы;
- Equality — оператор == и !=;
- CopyConstructor — глубокое копирование (изменение копии не влияет на оригинал);
- AssignmentOperator — корректная работа operator=;
- Union — операция | (объединение битов);
- Intersection — операция & (пересечение);
- Inversion — операция ~ (инверсия, включая корректную обработку «лишних» битов);
- StreamOutput — operator<< для вывода битовой строки.

Набор TSetTest (10 тестов):
- CreateAndMaxPower — проверка мощности;
- InsertAndCheck — добавление и проверка принадлежности;
- DeleteElement — удаление элемента;
- CopyAndEquality — копирование и сравнение;
- AddElementOperator — оператор + с элементом (исходное множество не должно меняться);
- RemoveElementOperator — оператор - с элементом;
- UnionSets — объединение двух множеств;
- IntersectionSets — пересечение двух множеств;
- Complement — дополнение;
- StreamOutput — вывод множества в формате { 0, 3 }.

Добавлен тест InversionZeroLength для проверки инверсии пустого битового поля (длина 0). Проверено, что при длине 0 не возникает ошибок сдвига, и результат остается пустым.

Результат: все 24 тест PASSED за ~0 ms. Это подтверждает корректность реализации как основного функционала, так и краевых случаев (пустое множество, выход за границы, «лишние» биты при инверсии).

Заключение

В ходе лабораторной работы были реализованы два класса:

1. TBitField — битовое поле с полным набором битовых операций, корректной работой с памятью (конструктор копирования, деструктор, оператор присваивания — правило трёх) и обработкой краевых случаев (выход за границы, «лишние» биты).

2. TSet — множество на основе характеристического вектора. Все теоретико-множественные операции (объединение, пересечение, дополнение, разность) сведены к побитовым операциям, что даёт высокую производительность и компактность.

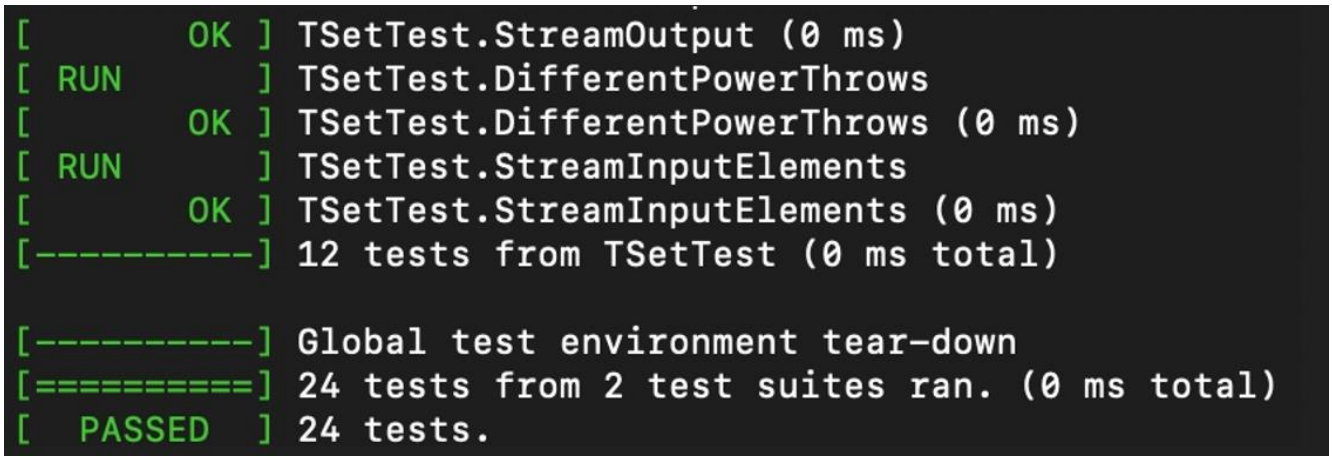
Освоены:
- принципы инкапсуляции и перегрузки операторов в C++;
- правило трёх при управлении динамической памятью;
- дружественные функции для перегрузки операторов ввода-вывода;
- фреймворк Google Test для написания автотестов;
- система сборки CMake с использованием FetchContent для подключения внешних зависимостей;
- базовые приёмы работы с Git и GitHub (fork, commit, push, pull request).

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Документация Google Test: https://google.github.io/googletest/
4. Документация CMake: https://cmake.org/documentation/

Приложение



https://github.com/IvlevAlexander0/3825B1FIsp1-2_3/pull/11